The Thiele Machine

A Computational Model with Explicit Structural Cost

Devon Thiele

Contents

The point 1

Before the book: the short version 40

I The picture 60

1 How I got here 61

2 What structure is 66

II The axiom 88

3 A record and a toll 89

4 Why the toll isn't arbitrary 103

5 Learning is free, committing is not 136

III The logic 148

6 The counter 149

7 Windows and shadows 161

8 Structural entitlement 181

9 What insight is 200

10 Records over any base: the latch 206

11 The whole direction 227

12 The shadow, exactly 235

13 The diagonal 241

14 Five small models 257

15 The pointer thesis 274

IV The machine 288

16 The smallest honest machine 289

17 Every model 374

18 Built from parts 381

19 Running it 387

20 The CHSH check 393

21 The counter against physics 413

V Checking it, and its edge 422

22 What Coq is, and why 423

23 How to break this 434

24 The claim ledger 442

25 Scope 470

26 Conclusion 476

Appendices 481

A Things I had to learn first 481

B Every assumption in one place 491

C What's mine, and what isn't 495

D Every result and where it is checked 506

E The exact words of twenty-one results 555

F The CHSH derivation, in full 579

G Constructing the quoting operation 601

H Sources and mathematical dependencies 604

The point

The strongest thing in this book first, so you know what the rest is for. On the small machine of Part IV, every program anybody can write, run from a clean start, raises its flag only after a passing check of a claim, a passing commitment to that same claim about the same edition, and then a passing certificate, in that order (Theorem 16.9). And one fixed program, the universal Thiele machine, runs every one of those programs on a host with more counters: it halts exactly when its guest halts, its flag goes up exactly when the guest's does, and its ledger equals the guest's at every guest step and, in between, never gets ahead of the guest's next one (Theorems 16.29 and 16.30). So the guarantee holds for every program, carried through one interpreter, with the cost counted exactly, and all of it is checked in Coq. Putting a rule under every program is an old idea, and I say whose later in this chapter. Carrying it through a universal interpreter with an exact ledger is the piece I didn't find done before, where I looked (Appendix C).

Here's the whole thing before any of the careful parts. I'm going to start with one small picture, take it apart, and keep asking what each piece means, as far down as I can get before I hit something that holds or something I can't get under. I'll tell you which. Then I'll put it back together at the end and you can check whether it still stands.

One warning about how this went, because it explains the shape of everything after it. Most of the time I had the hunch first and found the names later. Over and over, somebody had been here before me. Sometimes that made my day. Sometimes it pissed me off. You'll see both.

Two computers that end up the same

Put two computers on a table. Hand each of them the same list of a thousand numbers.

The first one sorts it. It compares, swaps, walks the list again and again until nothing is out of order. Then it does one more thing: it walks the whole list front to back and checks that every number is no bigger than the one after it. That check could have failed. It passes. Then it tidies up after itself, wipes its scratch work, and leaves the sorted list in memory. (That check compares neighbours, so what it settles is that the output is in order. That the output holds your thousand numbers and no others is a second claim, and it would take a second check comparing the output with the input. When I say sorted below, I mean in order.)

The second one never looks at your list. Somebody typed the sorted version into it earlier, same thousand numbers, same order, and it puts them in memory.

Now wait until both are done and look inside. Same bits in memory. Same bits on the screen. Set it up right, which you can, and it's the same everything, down to the last bit. One of them found something out about your list and stood behind it. The other one is holding identical bits and found out nothing. Wheel them out on a cart, swap them, wheel them back, and ask me which is which. I can't tell you. You can't tell me. And the part that bugs me most: built the ordinary way, neither machine can tell you either.

Sure, while they ran they were different. One did thousands of comparisons, the other did a copy. But the runs are over and the runs are gone, and nobody was being sloppy. Ordinary computers are built to forget how their bits got there, on purpose. Later in this chapter there's the sentence where a physicist said so in 1961, and why he said it costs heat.

Who cares? Anybody who's been handed a sheet that says "all tests passed." Anybody who's looked at an inspection sticker on a windshield: the sticker looks the same whether somebody checked the brakes or somebody sold the sticker. An audit, a proof, a ledger, an AI that sounds very sure of itself: every one of them is a pile of bits that could have come out of either computer, and the question that matters is the one the bits can't answer. Did something establish this, or did it just print it? I'll come back to every one of those at the end of this chapter and say which of them this book actually helps with. Fair warning: fewer than you'd hope.

I'm going to call the difference *established*. The first machine established that your list is sorted. The second has the same answer and established nothing. The rest of this chapter is me taking that one word apart.

Look harder

The obvious answer is: just look harder. Don't look at the end, look at everything. So make the computers as lookable as computers get. Scale them up to the size of a warehouse, every wire a lamp that's lit or dark, every switch something you can walk up to and touch. Nothing hidden in a chip too small to see. Put both of them side by side and walk the floor.

While they run, sure, you'd see the difference. One has a whole wing lighting up, comparing and swapping, and the other just copies. But they finish. The comparing wing goes dark. Now walk the floor and look at every single lamp. The sorted list is spelled out in lit and dark lamps in both, the same pattern, and everything else is dark in both, because both cleaned up.

There is no lamp anywhere that says "a check ran here and passed." Not hidden. Not encrypted. Just not there. You can see everything there is, and the thing I'm after isn't one of the things there is. Looking harder doesn't help, because the problem isn't that you can't see enough. It's that nothing was kept.

That's the shape I want you to carry through the rest: the problem was never visibility. It's keeping.

So keep the trace

Then the fix is obvious, isn't it? Don't let the run go. Keep the trace: every comparison, every swap, the check and its answer, written down as it happens. Wheel the computers out on the cart and back, read the traces, and you'll know which one sorted. Problem solved?

It's the right instinct, and it gets most of the way to this book. But take it apart, because "just keep the trace" has three things wrong with it, and each one is a piece of what comes later.

First, a trace is bits, and bits can be typed in. Whoever typed the sorted list into the second machine can type a trace in too: "compared, swapped, checked, passed," a few thousand lines of it, every one of them plausible. A trace written by a program is something the program produced, and the machine running the program doesn't care whether it's true. So keeping the trace doesn't answer the question. It moves it. Did something establish this trace, or did it just print it?

Second, the way out of that is to take the pen away from the program. Don't let the program write its own trace. Have something underneath it write the trace, one step at a time, as the steps actually happen, so nothing a program does can add a line that didn't happen or erase one that did. Airplanes do a version of this. Flight recorders go back to a prototype David Warren built in Melbourne in the 1950s. He was an Australian scientist at the Aeronautical Research Laboratories, working on jet fuel, and after a string of unexplained airliner crashes he wanted a box that would hold the instrument readings and the voices in the cockpit for whoever came looking afterwards; I know his story from museum and encyclopedia accounts, and I haven't read his own report. The recorders that came out of that idea are fed by the aircraft's own instruments. Nobody's account of events goes into them. And they're built to survive the crash, so the investigators don't have to depend on anyone's memory of what happened. That's the right shape, and it isn't mine. Later in this chapter it gets its name from computer security, a reference monitor, and I'll say outright that a machine built that way would be doing what this book describes. What the book adds is narrower than a rulebook for everything underneath. It's about the records that go up and never come back down: what that something has to keep for good, and what keeping it costs.

Third, you can't keep all of it. A full trace grows with every step and never stops growing, and a machine with finite memory runs out. Even the flight recorder doesn't keep everything. The cockpit voice recorder is a loop. I went looking for how long the loop is, and it has kept growing: for decades it held the last thirty minutes, an American rule in 2008 raised that to at least two hours, and big airliners built since 2021 carry twenty-five hours. Whatever the length, everything older gets recorded over. Every real machine that keeps a trace keeps a window of it, or a fingerprint of it, and forgets the content.

The fingerprint is the sharpest reply, so take it seriously. Fold every new line of the trace into a fixed-size scramble of everything before it, and you've got a hash chain. The timestamps of Stuart Haber and Scott Stornetta, two cryptographers at Bellcore whose story comes later in this chapter, work that way; so do the audit logs the cryptographers Bruce Schneier and John Kelsey designed in 1999; and so do the measurement registers in the security chip a lot of computers carry, the TPM. A running program can only extend those registers, folding a new measurement in. It can't write a value of its own into them, and the ones that matter here start over only at the next reboot. That really does keep something about the whole trace in a few dozen bytes. But look at what it keeps. It keeps a binding. The content isn't in it: to read the history you still need the trace stored somewhere, and the fingerprint only tells you whether a copy somebody hands you is the one that went in. The binding holds only because nobody knows how to find two traces with the same fingerprint,

which is a bet on how hard a search is. No law of nature backs it. That part isn't only my opinion. The computer scientists Moni Naor and Guy Rothblum proved a sharp version of it in 2005, and they even call the thing a fingerprint: keep a small private fingerprint of a big file somebody else stores, check the file by reading only a little of it, and against an opponent with unlimited computing the size of the fingerprint times the amount you read has to be on the order of the whole file. Do even slightly better than that against an opponent with limited computing, and you've shown that one-way functions exist, which is the hardness bet in its barest form. Their setting is a stored file. A growing trace is a different setting, but it's the same bet, with its terms written down. And squeezing traces that keep growing into a fixed number of bits has to land many traces on one fingerprint. That's a merge, hidden behind how hard it is to find which traces share a seat. It's the same merge as the people and chairs further down, where four people have to sit in three chairs. Fingerprints are great. They're not a way out of forgetting. They're a very clever way of forgetting.

So the real question was never "save everything." It's this: what's the smallest thing worth keeping for good, and what does keeping it cost? The answer this book ends up with is the commitment, in the database's sense, a question closed, and the trace drops away. (The cryptographer's sealed envelope also goes by the name commitment. It's a different thing, and both come up later.) Not every comparison the sort made. Just the one fact that a check of this very claim passed and the question got closed. And here's the part that matters most: on a finite machine, keeping even that one fact for good forces the machine to forget something somewhere else. That's the chairs, further down. And keeping things for good fills up, which Part II turns into a count.

So the premise was never that computers can't keep records. They keep records all day. It's that nothing in the rules of an ordinary computer makes the record paid for, permanent, or earned, and those three come in at different places, so here's where. Paid for is the model's one rule: in the Thiele Machine, the step that raises the record has to cost something. Permanent is a premise, the one the chairs need, and one of the conditions Part III puts on a record. Earned, meaning a check of this very claim passed first, isn't in the model's core at all. It's what the step of the small machine in Part IV enforces, proved for that machine and every program on it run from a clean start. One more word rides along with permanent, and it'll come up when I line my record up against other people's: dated. A record that goes up and stays up went up at some particular step. The first raising occurs at a definite step in the execution history. The latch itself does not store that step number. So dated, in this book, means dated in the execution history, and carrying a timestamp is a different property that the record doesn't have. A machine that charges one mark for every

I

The picture

1

How I got here

What I am is curious, past the point of good sense.

I pick a thing up and cannot set it down until I have turned it over enough times to see how it holds together, and this time it did not let go.

I am not a programmer and I was never formally taught any of this.

I was not trying to build a formally verified CPU, and the one I came close to building later, on the way here, is not part of this book.

I would not have known what one was if it bit me.

I was trying to build a 3D renderer.

There was no plan behind it.

Just a pull to make something real with my own hands, and the kind of stubbornness that will not let a thing go until it stands up on its own.

I have worked on it in off hours every day since.

Once I started digging in, two things came clear fast. I knew nothing about computers, and the field was already saturated, every obvious road paved and crowded. So I needed an angle nobody else was standing on, and the angle I had wasn't a strategy. It was just the only way my head works. I did not know what compositional meant. I had never heard of category theory. I called the thing I was building digital DNA, because that is what it felt like in my hands, and I thought about it in the only language I had: gravity pipes and directions and vectors, things that were linear and somehow not linear at the same time, structure that flowed and joined and came apart without my owning one proper word for any of it. I built a custom rendering pipeline out of those intuitions, by feel, throwing out pieces that did not sit right even when I could not have told you the rule they were breaking. And then, the way I always do, I went looking for what I was actually doing, and found out it already had a name. An old one. A deep one. What I had been reaching for in the dark was composition. It was category theory, a whole field I had never once heard of, sitting there describing the exact shapes I had been shoving around. The moment that landed, the rendering pipeline stopped being a rendering pipeline and started turning into a categorical engine, and I started swallowing the real theory as fast as I could find the words for things I had already built.

The method underneath was crude and it held. I wrote a long structured spec in JSON (a plain-text format for describing data) as a context document, fed it to Copilot, and had it build one Python file per concept, one at a time, each with its own tests, and I would not move to the next until I could watch the current one actually run. Category. Object. Morphism. Functor. Monoidal product. It came together in pieces and never in chapters, the way a picture resolves out of static. The Yoneda lemma (a result buried deep in category theory) was the piece that made the rest snap into place, the night I quit borrowing other people's runtimes and started writing my own around it, with Z3 (a constraint solver out of Microsoft Research) wired into the middle.

Then within weeks I found out the engine I had built to draw pictures was general enough to run a physics simulation on the exact same primitives, without my rewriting any of it. I wrote Newton's laws as a program in it, half expecting nothing, and it ran. I want to be careful about what that was and what it was not, because this is exactly the kind of thing that gets misread. It was not me deriving physics, and I am not claiming the machine explains gravity. It was the compositional structure being general enough to express a physics simulation the same way it had expressed a renderer, and that generality is what told me the structure was the point and the pixels never were. Somewhere in those weeks I crossed from debugging why one pixel would not show up to understanding that I had quietly built a categorical computing engine, and that the rendering had only ever been the first thing I aimed it at. The pixel did eventually show up, for what it is worth.

I kept going, because by then I could not not keep going. I built a small custom language for describing scenes (the name for that is a domain-specific language, a DSL), then a program that read the language and caught my mistakes in it (a linter), then a way to type commands at the whole thing from a terminal (a command-line interface, a CLI). The night the engine first ran a program declaratively, meaning I told it what should happen and let it work out the how without my spelling out each step, I told my wife, Julie, I thought I was onto something. She said okay.

Then I hit a wall, and it is the wall this whole document is about. Z3 could settle the small questions and fell over on the exact ones I needed answered, which left me staring down an infinite hallway of guess-and-test with no proof waiting at the end of it, ever. One night, not even working, I was watching a video about Flatland, the old story about a creature living in two dimensions who cannot so much as picture a third. And it landed on me sideways, all at once, the way the real ones always do. I said it out loud: I get it now. There is another axis. Classical computation only ever sees what it can read and write off its own surface, and there is a whole direction it is blind to. It isn't broken. It was just never built with an eye that points that way. That's how it landed that night, and it didn't survive checking in that form:

the direction turned out to be one an ordinary computer can reach, and what the usual rules leave out is the price (Section 3.6 says where the axis ended up). The two computers on the first pages of this book are my attempt to show you that blindness with something you can hold. I opened a new repo that same night and called the thing in it the Thiele Machine. The name began as a working title and became the name of the model. It carries the larger claim plainly: the machine treats structural state as part of computation.

What survived the whole trip from there to here is not the code. Not one line of it. By the time the Coq proofs finally landed, every line of that first renderer had been thrown out and rewritten at least twice, probably more, and good riddance to all of it. What survived was a single architectural commitment I made early and never once let anyone talk me out of: structure is the state itself, the actual thing the machine runs on. It is no metaphor laid over some realer state underneath. Category theory was the first language I found for that structure. It is not what the model is. The model asks for a state, a step, a record, a ledger, and a window, and category theory appears nowhere in that list. It lived on for a while in a bigger machine I built on the way here, a fifty-one-instruction virtual machine with its own chip design, whose partition graph (groups, and arrows between them) was, in different words and different syntax, the same object as the category structure that came together in that first week and first ran morphisms correctly at two in the morning. That machine was useful for building and confusing for everything else, so it's set aside now, kept but out of this book. What building it showed me is that six instruction kinds were enough for everything this book needs, and that the big one was mostly in the way. I did not write that code, not then and not now. I directed the models and refused everything they handed me until the morphisms composed the way the shape said they had to. The morphisms in that early code were objects that remembered where they came from; the records in the small machine I ended up with remember where they came from too, and you can walk up to one and challenge it. The receipt grew teeth. But the shape itself never moved. The shape was the substrate the entire time, and everything else, every renderer and language and proof, was me slowly learning to see what I had already built. By the shape I mean structure held as state. Not category theory. Category theory was just the first language I had for it.

Formal verification was a graveyard of dead ends further on. The hardware came after that, in the bigger machine I set aside, and none of it is in this book. Every step pulled back to show another layer underneath that I had not known was there, and I mean that plainly: I did not know what a Turing machine was when I started, I did not know what Coq was, I did not know most of the words in this document existed. I learned each one the same way, by walking face-first into a wall that needed it and not being let past until I had it. The needing always came first. The learning came after, dragged along behind the needing because it had no say in the matter.

Large language models were my hands the whole way through, and I want the division of labor exact, because it is the thing people most want to get wrong about work like this. The hands had nothing like this to draw on: no reference implementation, no prior project to point at. The ideas, the architecture, the direction, the demand that the thing be true: those are mine, all of them, no asterisk. The symbol-pushing, the actual Coq and the actual math, the models produced and the proof checker judged. The proofs compile or they do not. I refused every one that did not. Coq is an obedient idiot. It checks the proof matches the statement. It does not check that the statement means what you think it means. A theorem can claim "every cat that is also a prime number has whiskers" and the proof can be three characters long and Coq will nod and stamp it. So I read every statement. The proofs you can't fake. The statements you can fake by accident, which is the more dangerous failure mode. I refused both kinds. However the symbols got onto the page, the argument on it is checkable by reading, and it would be exactly as right or as wrong if a million monkeys had typed it. The monkeys could not have refused.

A word on what I actually do, because it runs under everything after it. I do not write code or math by hand. I do not pick constants. I do not grind through derivations. I have never written a line of code on paper or in a notepad in my life. What I do is see, and direct, and refuse. When I take hold of an idea it does not stay a point. It opens up. It becomes a shape I can blow up and turn over and take apart, lay across another shape to find where the two of them touch, run forward and then back to where it started, zoom into until it is the only thing in the world and pull back until it is one dot in a field of other dots, and the whole time it stays wired to everything I have ever connected it to, the right connections and the wrong ones both, all of them live at once. I am not reading that field neutrally, the way a search would. The thing doing the looking is me, the whole of me, my gut and my mood and my stubbornness picking what lights up and which way I move through it, and that is the part you cannot lift out without turning me back into a search engine. So I see where a thing has to fit, the shape it is obligated to have, usually well before I could write it down in a single symbol. Then the model gives me the symbols, the proof checker takes them or throws them back, and I refuse anything that will not compile or does not match the shape I am holding. That refusal is the one thing in

this entire document that is mine and only mine: the demand that it be true, and the throwing-out of everything that wasn't.

I am telling you all of this because it's the engine. I did not come up through anywhere that hands you a foundation and says trust us, this part is settled. No professors, no colleagues, no field full of authority to lean back on when a claim got heavy. From the outside that is a deficit. From the inside it is the whole reason any of this is solid: when I meet a claim I either find the proof or I do not believe it, and that goes double for my own, because my own are the ones I most want to wave through. It is how I am built. It is the only way I know to hold a thing and actually trust that I am holding it.

So everything in here gets built from the ground up. It isn't a courtesy to you. I build it because I do not believe I understand a thing until I have built it with my own hands, watched it stand up on its own, and tried to knock it over. I built this document the exact way I built the machine, brick by brick, refusing each brick until it held weight. That includes the parts you already knew cold before you opened this. I did not skip them for you, and I could not skip them for me.

The vocabulary I had to go look up to do any of this sits at the back of the book, so the argument comes first. If a word stops you, it is there. If you already know this material, you never have to open it. But if you catch something in the basics that is wrong, I want to know, and I mean that as a real ask. It isn't politeness. Nothing in here is correct just because I wrote it. Especially the parts I wrote.

On the name. I wanted to call it the meta machine. Then, in October 2021, Facebook renamed its company Meta: a company taking a term for self-reference as its name and draining the word of meaning in the same motion. So I went one level up, where this machine already lives. The most meta move available, for a machine that runs on meta-level reasoning, was to put my own name on the cover and take the word back. That's the whole of it, a meta move on a meta machine. It happens to be my dad's name too, and my wife's now. So I asked them before I did it. They said go ahead.

Then Section 10. Section 10 said: take any base, and any record that answers only to the base and never comes back down, and the record is a latch. This machine has a base: a program together with a core, stepping by the program step. And it has a record, the flag, with the ledger riding along. So the question is whether it's an *honest* extension in Section 10's five-rule sense (Definition 10.2), and if it is, Section 10 does the rest.

Theorem 16.7 (The flag is a latch of the base)**.** Let the base have states (program, core), stepping by the program step, starting at start(a, b)'s core for any a, b, and halting when the program has halted. Map each (program, state) to (program, core). Then the machine, run as a closed machine on (program, state), is a base-driven record over that base, and so its flag factors as the latch (Observation 10.1) of the event "the next instruction is a CERTIFY that passes."

Proof. The rules of Definition 10.2 one at a time. Driven by the computation: the next flag is the old flag or "the next instruction is a passing CERTIFY," which is a function of the base state and the old flag (that's the step line). Ledger: μ only adds. Toll: Theorem 16.5. Permanence: Theorem 16.5. A reachable write: the three-line program of the worked run raises the flag from a start. Section 10's theorem then gives the factorization. □

Checked as `earned_core_honest` and `earned_core_is_latch` (EarnedCoreLinks.v), the latter by `record_axis_is_latch_holds`; the step-level facts are `cert_latch` and `base_blind` (EarnedCore.v).

So far, nothing new. The toll, the floor and the latch are things Parts II and III already promised about any machine of this kind. The next subsection is the part neither of them could give.

16.6 Commitments are earned

Here's the shape before the words. Pick any run from a clean start, any list of instructions at all, chosen by somebody who'd love to cheat. Look at the moment a COMMIT would pass. Then, somewhere earlier in that run, there's a CHECK of exactly the same claim that passed and paid, at the same edition of the counter, and between that CHECK and now, nothing touched the counter. Not "probably." Not "on the programs I wrote." On every list of instructions anybody can write.

To say "nothing touched it" exactly, one word.

Definition 16.5 (Untouched)**.** A list of instructions *mid*, run from a state s, leaves counter c *untouched* when, for every way of splitting *mid* as t_1, then i, then t_2, executing i at the core reached by running t_1 from s leaves both $\mathrm{ver}(\cdot, c)$ and $\mathrm{val}(\cdot, c)$ as they were.

Defined as `untouched` (EarnedCore.v).

Theorem 16.8 (A commitment was checked first)**.** Let s_0 be clean and *pre* any trace after which COMMIT p c would pass. Then *pre* splits as pre_1, then CHECK p c, then *mid*, where: the CHECK passes at the core reached by pre_1; CHECK p c and COMMIT p c each cost at least 1; c's version after pre_1 equals its version after *pre*; and *mid*, run from just after the CHECK, leaves c untouched.

Proof. COMMIT passes, so the claim $(p, c, \mathrm{ver}(c))$ for the current version is in the table. By the no-forging invariant (Theorem 16.12 below), that fact was written by a passing CHECK p c at some point pre_1, with the version c had then; since it's the current version too, the two versions are equal. Versions never go down, and any instruction that changes a counter's value also raises its version, so an instruction in *mid* that touched c would have left c's version higher at the end than at the CHECK. It isn't. So nothing in *mid* touched c. □

Checked as `earned_commitment_provenance` (EarnedCore.v).

Theorem 16.9 (A raised flag was checked, then committed, then certified)**.** Let s_0 be clean and t any trace with $\mathrm{cert}(\mathrm{run}(t, s_0))$ up. Then for some p and c, t is

$$pre_1,\ \text{CHECK}\ p\ c,\ mid_1,\ \text{COMMIT}\ p\ c,\ mid_2,\ \text{CERTIFY},\ post,$$

where the CHECK passes at its state, the COMMIT passes at its state, the CERTIFY passes at its state, c has the same version at the CHECK as at the COMMIT, and mid_1 leaves c untouched. And when the CERTIFY runs, the channel holds exactly the claim that COMMIT made: the fact (p, c, c's version at the CHECK). CERTIFY takes no operand; this is what ties it to the claim.

Proof. The flag starts down, so some first instruction raised it, and by Theorem 16.5 it's a passing CERTIFY. A CERTIFY passes only with a fact in the channel, the channel starts empty, and only a passing COMMIT puts a fact there, so some earlier COMMIT p c passed. Take the last one before the CERTIFY: the fact in the channel is the one it wrote. Theorem 16.8 at that COMMIT gives the CHECK, the equal version, and the

untouched stretch, and the equal version makes the committed fact (p, c, c's version at the CHECK). □

Checked as `earned_certification_provenance` and, with the channel clause, `earned_certification_same_claim` (EarnedCore.v).

And the failure side, so you can see the refusals are clean, with nothing left half-done:

Theorem 16.10 (Unearned steps trap and change no claim)**.** If COMMIT p c would not pass at a core k, executing it puts the trap up and leaves the channel and the table as they were. If CERTIFY would not pass at a state s, executing it puts the trap up and leaves the flag as it was.

Proof. If the trap is already up, the core is unchanged and its trap remains up. Otherwise, the failed COMMIT takes the failure branch of the instruction table: set the trap, leave every other core field alone. A failed CERTIFY takes the same branch, and its fires value is false. The flag update is then its old value or false, which leaves it unchanged. The instruction's charge still applies to the ledger. □

Checked as `unearned_commit_traps` and `uncommitted_certify_traps` (EarnedCore.v).

If that rings a bell, it should. It's Milner's move from the opening chapter: in LCF a theorem is a value that only the rules of the logic can build, so nobody can just print one (Milner and two colleagues on his Edinburgh project, Michael Gordon and Christopher Wadsworth, wrote the system up in 1979, and Gordon, who went on to build the HOL prover at Cambridge, told its history in 2000). Here the thing nobody can print is one bit, and what guards it is one step rule. Milner's guard was a type checker. Same architecture, much lower down.

A provenance theorem is also an itemised receipt, and the itemised receipt has a history that's exactly this gap. James Ritty ran a saloon in Dayton, Ohio, and believed some of his staff were pocketing money. On a steamboat trip to Europe in 1878 he watched a mechanism that counted the turns of the ship's propeller, and wondered whether something like it could count the money instead. In 1879 he and his brother John patented a "Cash Register and Indicator," and they sold a version under the wonderful name Ritty's Incorruptible Cashier. The first model was this book's counter (Section 6) with brass on it: press a key, the amount goes onto a dial showing the day's running total. No drawer, no paper. At closing time you hold the dial up against the cash. And that's all a total can do. It tells you *that* the money is short. It can't tell you which sale went missing or who rang it up. The paper came

later. The patents passed to other owners, first a Cincinnati salesman named Jacob Eckert, and in 1884 to John H. Patterson, who renamed the business the National Cash Register Company, and the story told about that company is that it added a paper roll: one copy kept for the shop's own books, one handed to the customer. That last part I only have from histories of the company and of the machine, so hold it the way you'd hold any company's story about itself. The order is right either way: first a total that can tell you something's wrong, then a list that tells you where. The ledger μ is the dial. Theorem 16.9 is the list: which CHECK, of which claim, about which edition, before which COMMIT. The theorem supplies that list about the run. The machine doesn't keep it as data anyone can read back from the end state, and nothing in the theorem gives a procedure for rebuilding it from there. A fact in the table holds a property, a counter and a version number. That's neither the counter's value back then nor a log of what happened in between.

Read Theorem 16.9 once more for what it *doesn't* say. Between the CHECK and the COMMIT, the counter is untouched. Between the COMMIT and the CERTIFY, it isn't promised anything. You can check A is even, commit to it, add one to A, and then certify. The flag goes up, and what it stands on is a commitment about the edition that was current when you committed. The certificate is historical: it says "this was checked and committed about this edition," and it says nothing about whether that's still true. I think that's the right meaning for a certificate (a birth certificate doesn't expire when the baby grows), but it's a meaning, and you should know which one you're getting. So a certificate doesn't say the claim holds at the CERTIFY, and it doesn't say the claim holds at the end of the run. A CERTIFY that rechecked the version would give a different guarantee. That's a possible extension, and the historical theorem doesn't need it.

16.7 What a passing check means

Provenance says a CHECK happened. It doesn't yet say the CHECK was *right*, and that's a separate question with a separate proof. A machine could have a perfectly ordered chain of check, commit, certify, all standing on a checker that says yes to everything. So: what does a fact in the table actually tell you about the counter?

Theorem 16.11 (Checker soundness)**.** Let s_0 be clean and t any trace, and let k be the core of $\mathrm{run}(t, s_0)$. For every fact (p, c, v) in k's table with $v = \mathrm{ver}(k, c)$, the property holds of the counter's current value: $\mathrm{holds}(p, \mathrm{val}(k, c))$. In particular, if COMMIT p c would pass at k, then $\mathrm{holds}(p, \mathrm{val}(k, c))$.

v

Checking it, and its edge

22

What Coq is, and why

Coq is a proof assistant, which is a phrase I had to go and look up. Underneath the phrase it's a programming language, but one whose type system is strict enough to do a thing ordinary languages can't: you write a proof down as a program, and the compiler refuses it unless the program genuinely matches the statement you claimed. You can't talk it into a maybe.

The name is a joke three ways at once, and it's worth knowing before you say it out loud in English. *Coq* is French for rooster. It's also a nod to Thierry Coquand, a French computer scientist and mathematician, at the University of Gothenburg as of 2026, who worked out the logic it started from with Gérard Huet, a French computer scientist at INRIA, and to that logic's initials, CoC, for the Calculus of Constructions; and there's a French habit of naming research tools after animals. The English slang reading did what you'd guess. By the development team's own account, some people, women students especially, ended up never learning the tool because of its name. So in March 2025 it got a new one, the Rocq Prover, after Rocquencourt, the Inria site where it was first built, and after the Roc, the giant bird of legend, so the bird stayed. Everything in this book was checked with Coq 8.18, from before the rename, so that's the name I use. The rooster changes nothing about what got checked.

22.1 Types as claims

Here's the idea, and it's one idea. In a normal language a type says what kind of value a variable holds: in Python, "x is an int" means x is a whole number. In Coq, types are *claims*. The type "n is greater than zero" is the type of *proofs* that n is positive. A function from "n is greater than zero" to "n plus one is greater than zero" is a proof that if n is positive, n plus one is too, and the compiler checks that the function really has that type, the way it would check that a function from integers to strings really returns strings. This correspondence between proofs and programs has a name, Curry-Howard, after two people who noticed it separately decades apart, and Coq's foundation, the Calculus of Inductive Constructions, is a type system built to exploit it. Coquand worked it out with Christine Paulin-Mohring, a logician and computer scientist best known for developing Coq, on top of the Calculus of

Constructions he and Huet had made; their paper on it was given in 1988 and printed in 1990, so the late 1980s is as close as I'd date it. The "inductive" part means you define your own recursive types (numbers, lists, trees, machine states) inside the system. The numbers, lists and states this book uses are all defined that way, inside the system, so the foundation is explicit; nothing is smuggled.

"Decades apart" undersells it, and the gap is worth a side trip. Haskell Curry, an American logician at Penn State best known for combinatory logic, noticed in 1934 that the type of a function from A to B reads like the claim "A implies B," and that the types his functions could have lined up with the claims you can prove; he and Robert Feys, a Belgian logician at Leuven, pushed it further in 1958. William Howard, a proof theorist at the University of Illinois at Chicago, wrote the sharper version in 1969, and then it went around as a photocopied manuscript until 1980, when it was finally printed in a collection of essays for Curry. (The Edinburgh computer scientist Philip Wadler's 2015 survey, "Propositions as Types," tells this story.) So the sharpest statement of the idea under the tool this book is checked with spent its first eleven years as a stack of photocopies. Is that the bottom? No. Reading a proof of "if A then B" as a method that turns any proof of A into a proof of B is older still: the intuitionists had it in the 1920s and 1930s, Brouwer, Heyting and Kolmogorov, which is why people also call it the BHK interpretation. Where it really starts I can't tell you. It changes nothing in what the checker checks. It changes how I'd tell the story: nobody sat down one day and invented "proofs are programs." It kept getting noticed, by people coming from different directions, until somebody wrote it down plainly.

The compiler checks whether the types line up. If your proof has a hole, it rejects the proof, unless you write "trust me" (the keyword is Admitted), and then it marks the theorem as unproved. But here's the part that matters: the compiler checks that the proof matches the *statement*. It does not check that the statement says what you *meant*. Hand it a statement that means nothing and it'll prove it for you, three characters, no complaint. So a proof being accepted is not the same as the thing being true. It means this proof matches this statement, and whether that's the right statement is on me. The machine can't tell. The point said a short version of this and I'm saying it again because every section since has leaned on it: the checked names in this book are a promise that the *proof* is there, and the sentences around them are my claim about what the statement *says*, and the second can be wrong while the first is fine, which is why so many sections of this book stop to say exactly what a statement's hypotheses are.

22.2 Why I used it

Not to make anything true. I used it because I don't trust my own eye to catch a gap in an argument I want to believe. I spent months convinced by my own reasoning, pushed the proof through, and watched a key step come apart in my hands. The machine caught where I'd fooled myself. That's what it's for. Gap-catcher. Not the proof. The proof is the argument, and an argument has to close in a way a person can follow, or it hasn't closed at all, which is why this book is written in prose. A list of theorem names can't close an argument for anyone.

Two precedents show that a proof assistant is a serious tool. Neither of them is a toy. CompCert, Xavier Leroy's C compiler, written and proved correct in Coq: the assembly it emits behaves as the source does, for programs with defined behaviour. And Georges Gonthier's 2005 proof of the four-colour theorem, a century-old problem whose first proof (in 1976) needed a computer to check fewer than two thousand configurations, by its own announcement's count, and was distrusted for exactly that reason. Gonthier redid it in Coq, working from a later version of the proof by Neil Robertson, Daniel Sanders, Paul Seymour and Robin Thomas (1997), so that the *checker* was the small, auditable thing and the cases were the program. That's de Bruijn's criterion applied to mathematics itself: the thing you have to trust is the small kernel. The big proof only has to get past it. The name was given by Henk

Barendregt, a Dutch logician known for his work on the lambda calculus and type theory, after Nicolaas de Bruijn, whose Automath project, begun in 1967, kept the part of a proof checker you have to trust small: a system meets the criterion when its proofs can be checked by a small independent program that a person could read.

The 1976 proof deserves its own side trip. Kenneth Appel and Wolfgang Haken did it at the University of Illinois, and the mathematics department changed the slogan on its postage meter to "Four colors suffice." The slogan it replaced had been on the department's mail since soon after 1963, and it said that $2^{11213} - 1$ is prime, a prime Don Gillies, a professor at the same university who helped design its ILLIAC II computer, had found in 1963 while checking that machine out. Think about that for a second. For years on end a mathematics department stamped its mail with results nobody could have checked by hand, first one and then the other, and both were right. So was the distrust of the 1976 proof wrong? I don't think so. The worry was never that the computer lied. It was that nobody could read what had been checked, and a check you can't look at is hard to lean on. Gonthier's answer was de Bruijn's: make the part you have to read small enough to read.

22.3 What "zero Admitted" means

"Zero Admitted" is literal. Nowhere in the proof tree did I write "trust me" and walk on. Every step the checker could see, it saw, end to end, with no "and the rest is obvious." It's a hygiene fact about the gap-catcher. It passes no verdict on the argument: it says I didn't let myself skip a step where the machine was watching. It does not say the statements are the right statements.

What it doesn't mean, listed. It doesn't mean the theorems prove physical reality; they prove things about the machine as formalised, and whether that describes the world is answered by experiments (Section 4, Section 21). It doesn't mean Coq's own foundation is proved consistent; it's believed consistent, and that can't be proved from inside, by Gödel (Section 13), so the foundation is trusted, and "trusted" has substance: it's the same kernel CompCert and the four-colour proof stand on, and a bug in it would take all of them down together, which is a kind of safety in numbers. And it doesn't mean no axioms: the proofs use a handful of standard-library assumptions, and the assumption audit names them: functional extensionality on dependent functions (two functions that agree everywhere are equal); two decidability principles that come with the standard library's real numbers, used wherever real analysis is, for example in the matrix proofs behind the game check and in the real-valued results of Sections 4, 12 and 21; and, in some theorems, the law of excluded middle (every proposition is true or false). The count, from the full audit: the full-corpus probe covers 5,997 named theorems across 341 files: 5,198

close under the global Coq context outright, and 799 use only Coq standard-library assumptions. Zero project-local axioms appear in any of the 5,997 dependency trees. The small machine's files, under minimal/, are inside that probe too, and every one of their 1,417 probed theorems comes back closed under Coq's own rules, with no axiom at all. Among them are the small machine, its version for any property language, the simpler host, the definition of Thiele-complete and its window theorem, the host machine with a register for every whole number, the numbers that write a guest program down, and the pigeonhole (Section 16), and the files that run Parts II and III on that machine and the diagonal on its programs (Sections 16.17 and 16.18). Each also ends by printing the assumptions of every headline theorem in it, and every one of those lines says the theorem is closed. A lemma a closed theorem uses is closed with it. So for those files every headline theorem is closed under the global context, resting on no axiom of mine or of the standard library, and two separate printouts say so. That isn't the same as having no premises: each theorem keeps the hypotheses written in its own statement. Only two small files under minimal/, MuCore.v and Napkin.v, sit outside the big audit, and they are built and checked on their own by `bash scripts/verify_core.sh`, which compiles each from a clean slate with `coqc`, fails on any `Axioms:` line, and expects ten closed theorems from MuCore.v and seven from Napkin.v. The universal program's own files, which lean on the vendored library's counter-machine blocks, sit in the kernel tree with the other link files, and they also end by printing their headline theorems' assumptions; so do the files of the priced host, the compiler from computable functions and the theorem for every computably presented Thiele machine. In plain words: every theorem in the audit either uses no axiom at all or uses only those standard-library axioms, none of mine. The hypotheses in each statement are a separate matter, and they stay. Those counts are of the sources at this edition. Add a theorem and the counts move. The shape, no axiom of mine anywhere, is what every edition has to keep.

22.4 What you're trusting, in one place

If you accept every checked theorem in this book, here is what you're still taking on trust, all of it, so you don't have to collect it from five chapters.

1. The statements. The checker confirms a proof proves its statement. It doesn't confirm the statement says what my sentence about it says. That's why the book prints the important ones in full, premises and all, and why Appendix E prints the Coq text of the load-bearing ones beside a line on whether my sentence says exactly that, more, or less. A machine check supplements that reading.
2. Coq's kernel and its foundation, and the standard assumptions above.

26

Conclusion

I started with one word and a machine I couldn't explain. Here's where the word ended up, and where the machine did.

Structure, by the end of Part I, was relations among things, kept under chosen changes, carried by whatever stuff, always relative to a frame. The start I chose, the smallest one by the count of names, was a sorting of a pile, and the thing nobody I read had priced was the moment you stand behind the sorting. That was Part I, and it ended with a question.

The question got an exact form in Part II: five parts, one rule, one theorem you proved at the table. Then the rule stopped being only my preference. With finitely many yes-states and a flag that stays up, every move that raises it merges states, and if forgetting is charged for, the way Landauer's principle charges for it in its worst case, the toll follows. That's a derivation under three premises you can refuse one at a time, each with its counterexample built. It isn't a proof that every computation has to adopt the rule; and the charge on forgetting is a named premise with its critics named. Learning can be free; committing to a record that stays put can't be, on those premises. That's as far as physics goes in this book, and I've tried to keep it exactly that far.

Part III was the logic, and here's what it earned, in the order it was earned. The counter is arithmetic once the schedule is chosen. A window in which a raising step and a quiet one look the same can't price the raise exactly, by a fact about functions far older than the machine. Entitlement is defined as an object, a checked relation, and a paid event; given lists, a tree and a covering somebody supplies, a run that paid a charged step for every level of the tree has paid for the index bits it commits to (no run is shown to have to pay them, the run doesn't recover the tree, the lists count repeats, and the index bits aren't the questions anybody asked); on every Thiele-complete machine the same count, for a run from a clean start, is a theorem about its record moves, with the earned chain inside the run, and the count can't do without the covering; and it is a license to lean on a claim. It doesn't prove the claim true. Insight, in the sense the toll prices, is a committed relation. A feeling doesn't count. Its floor is flat: at least one, however deep. The claim behind the flag is something a check could have refused, and in the bare model nothing makes the commit wait for the check. Over any base, a base-driven record is a latch, with

two free choices, the event and the price list; "only one substrate" is false and "one structure, many price lists" is true once the event is fixed. No procedure the substrate itself can represent decides the shortcut predicate, given a recursion theorem, by the same diagonal that broke Frege's floor, and the machine's answer is to record the commitment. It leaves the judgement unautomated. Of five record disciplines I modelled in fragments, one charges a fee at the write. And which event to record is a thesis, with its refutations listed.

Part IV built one small machine, two counters and six instruction kinds, with hosts to run it, and on it the model stands whole: from a clean start, a raised flag was checked, then committed, then certified, on the same edition of the same counter, nothing gets forged, the cheapest run to a raised flag adds exactly three to the ledger, and a run whose check could have failed reaches that three (the step doesn't police trivial checks), and it computes everything Minsky's two-counter machines can, so its halting is undecidable. Earned commitments and universality are proved there. So is what Thiele-complete has to mean: a universal base that's free and can't touch the record, a record that stays up once it's up, an earned record, one mark for each check, commit and certificate, and some claim whose check can fail. Every clock, every machine that charges for computing, fails it; the small machine meets it; and every machine that meets it hides its record and its ledger from its own window. And so is a universal Thiele machine, with its scope said plainly: one fixed program, U, written in the small machine's own six kinds of instruction on a host with more counters, with no guest step built in, that runs every small-machine program from every clean start start(x, y), halts exactly when its guest halts, raises its own flag by its own check, commit and certificate if and only if the guest's rises somewhere in its run, and keeps a ledger equal to the guest's at every matching point. And then the bigger version of the same thing: with one paid move added, one fixed program, U_P, runs every computably presented Thiele machine, a machine from anywhere whose driver, step, prices and reading are computed by programs. It halts when the guest stops, raises its own flag on its own earned chain exactly when the guest's reading has said yes, with the guest's own reading as the claim it checked, and, when the guest's reading starts at no, keeps the guest's prices to within a surcharge of at most two, which no program on that host, run from a clean start, can avoid when the guest certifies with its whole ledger below three. An ordinary programmable computer, as built, doesn't do that for the programs it runs; U and U_P show a fixed interpreter can. All of that is proved in Coq with no axioms: each file prints the assumptions of every headline theorem, and every line comes back closed.

Then a check worth earning, the two-player game with an integer test whose pass forces Tsirelson's bound for the tally it's handed. That one does use axioms, the standard library's: functional extensionality and the two decidability principles of

its real numbers, because the bound is a statement about real numbers. And what the counter has to do with physics, which is less than the word Landauer in a file name suggests. The places where the machine came up short of the idea stayed on the page: its free steps merge states, so the chairs don't force the toll on it, and only the merge premise asked of the record layer does; and a check that can't fail certifies for three like any other. Part V was how to check it and how to break it. Every result carries the shape of the thing that would kill it, gathered in one place (Section 23): a state of the small machine and an instruction after which the ledger goes down; a contractive correlator whose pinned matrix isn't positive semidefinite; a certification system that follows the toll and certifies at total cost zero; a function on numbers that correctly decides the shortcut predicate of the substrate built from it. Any one of them brings down the theorem it matches. I hunted. I came up empty. For a theorem Coq has checked, that's no surprise: finding one would take a bug in the checker. The places a reader can really win are where my sentence says more than the statement does, or a definition doesn't catch the word it's named for, and Section 23 starts with those.

If you asked what's new here, I wouldn't give you one thing, because there isn't one. There are four, and each comes with its fence and its neighbours. First, the toll isn't only my preference: on a finite machine a reading nothing turns off can only be turned on by a step that merges states, so if merges are priced, raising it costs (Theorem 4.1, Observation 4.2). The counting under it is the finite cousin of the recurrence argument Zermelo threw at Boltzmann in 1896, and the price on a merge is Landauer's, as a premise. Ouldridge and Wolpert (2023) had already put a physical price on that merge for finite automata, so what's here is the toll as a rule on a ledger. Second, a window that shows a raising step and a quiet one the same can't price the raise exactly (Observation 7.2); that a price computed from what a window shows can't tell apart two steps the window shows the same is elementary, and what's here is the pricing form. Third, a record driven by the computation that never switches off is a latch on one event, over any base (Observation 10.1); latches and history variables are old, and the theorem covers only records that never come down. Fourth, a record that's priced, and on the small machine earned (Theorem 16.9), under the move of guaranteeing something about every program. That move isn't mine at all. It's the reference monitor's (Anderson, 1972), made precise for monitors that watch a run (Schneider, 2000), and verified in Coq for monitors checked at every instruction (Azevedo de Amorim and coauthors, 2015). What's put together under it here is built and checked, and its idea isn't new: the check bound to the edition it looked at, the toll paid on top, and universal programs that carry both to every program of the small machine and to every computably presented Thiele machine, earning their own flag as they go. Appendix C sorts all four with sources.

And two things I'd point at if you asked how far the checking reaches. Neither is a fifth new thing.

In the game: an integer check on eight counts forces a five-by-five matrix to be positive semidefinite, and from there the Tsirelson bound follows, and the chain never once reaches for physics. No Hilbert space, no operators, no observables. The route through the matrix is the first rung of a ladder Navascués, Pironio and Acín built in 2007. None of that route is mine, and the semidefinite road to Tsirelson's bound is older than that. What's here is the check that runs in whole numbers, and the checked link from "the check passed" to the bound. It's algebra you can follow line by line, and the assumption audit on that link finds exactly three standard axioms from Coq's own library (two that come with its real numbers, and functional extensionality) and nothing of mine, which is how anyone can see that no physics was smuggled in. And on the small machine's host a raised flag can stand on that check: from a clean start a raised flag contains a passing CHECK of the packed counts, a COMMIT and a CERTIFY, and the check passes exactly when every pair was sampled and the five-by-five is positive semidefinite (Section 20).

In the model: the diagonal rules out a correct decider whenever its flip can be expressed inside the substrate and the substrate supplies the fixed point; in L the fixed point is proved from the reduction rules and the diagonal is Rice's theorem. On the small machine's host, Kleene's recursion theorem and Rice's theorem are proved for its own programs, so the wall stands there by proof, with its premise a theorem: no host program decides a property of what a program computes that some program has and some lacks; such a property is undecidable in the library's sense (Section 16.18).

What this book doesn't settle, by my count. Whether the toll is the *right* thing to price. The five real systems mostly built a wall or a penalty for contradiction. Whether certification is the pointer event. Whether Landauer's critics are right, which would leave every theorem standing and one premise unowned by the world. And the question I'd most like someone to take from me: is there a smaller start than a partition, by a ruler they can state? I said in Section 2 it would change this book. It would.

And the two computers from the first pages, the sorter and the copier, holding the same bits. Walk the warehouse again. On an ordinary machine there's still no lamp for what the sorter did, and nothing in this book changes that. On the small machine there is one: a flag and a counter you can read off the state, and a theorem that, from a clean start, the flag can't be up unless some claim went through a passing check, a commit about the same edition and a certificate, in that order, where a claim can be any property, in a language with an exact equality test, whose checker is proved to mean what it says, "this counter, read as a list, is sorted" among them (`sorted_run_certifies_iff` and `sorted_run_refused_forever`, EarnedGeneric.v). That's the one lamp I set out to light. It's a lamp for whoever can read this machine's state. A "tests passed" sheet fits it only when the test is a checker proved to mean what it says; the sticker and the very sure AI from the first pages ask something else, whether to believe a report you didn't watch being made, and nothing in this book answers that (Section 25). Whether every machine ought to have that lamp is the argument. No proof here settles it.

So that's the Thiele Machine. An abstract model with five parts and one rule, an argument made with it that the usual account of a computation is a shadow and the axis it leaves out belongs in the step, and one small machine to check the argument against, with one fixed program, written in its instruction kinds plus one paid move and run on a host with more counters, that runs every machine of the model whose workings a program can present. The theorems are what they are, with their premises printed. The interpretation is mine, and I've tried to say at every turn which is which, because the first time through I was the only one checking, and I've met me, and that's not enough people. Now there's you.

A

Things I had to learn first

I came in with none of the formal fields this work turned out to need. By the time the proofs compiled I'd needed enough of half a dozen of them that I had to put down somewhere what those fields had words for. This appendix is that place. None of it is the argument. It's what I needed to be able to make it. Everything below is what I understood after I went and read about it; I'm not claiming to teach you these subjects, and if you already know a part, skip it.

A.1 Bits, bytes, and what a CPU sees

A bit is one digit that's either zero or one, the smallest fact a machine can hold. Everything else is bits stacked up. Eight of them make a byte, which lands you any number from 0 to 255, and the first thing that genuinely surprised me is that the same byte is also a letter: a convention called ASCII simply agrees to call 65 the letter A, 48 the digit 0, 32 a space. Nothing in the byte knows which it is. The meaning is a convention laid on top.

You can see the convention's seams with a pencil. Write 65 in binary: 1000001. Knock off the first two digits and what's left, 00001, is 1, and A is the first letter. B is 66, which ends in 00010, two. Now lower case: a is 97, which is 1100001, the same pattern with one more bit switched on, the one worth 32. Every capital and its small letter differ in exactly that bit. That's on purpose. The first version of the code, in 1963, had no lower case at all; when lower case arrived, in the 1967 revision, it was placed so the two cases differ by one bit, which made matching them, and building keyboards and printers, simpler. Nothing in the byte knows it's a letter, but somebody made the letters easy for a circuit to tell apart.

Everything in this subsection is about real CPUs. None of it belongs to the abstract model, which has a kind of state and a step rule and nothing about widths or slots, and the small machine of Part IV needs very little of it: its two counters have no width at all, and hold any whole number.

A word is a chunk of bits the CPU treats as one unit, in 2026 usually 64 bits (0 to about 18 quintillion) on a desktop or server and 32 (0 to about 4 billion) on many small devices. The abstract model doesn't care which width.